MITRC

MODERN INSTITUTE OF TECHNOLOGY AND RESEARCH CENTRE, ALWAR

Name of faculty: Dazy Arya

Subject & Subject Code: CF&P,1FY2-08

Semester: II **Session:** 2021-22 (Even Sem.)

Branch: AI&DS, ME, CE, EE **Batch:** C,D

UNIT: I **Date of Submission:** 28/03/2023

Lecture No.	Topics Covered	Total Page
1	Fundamental of Compueter stored programe Architecture of Computer	2
2	Types of Computer	3
3	storage device-primary memory and secondary storage,Random,Direct,Sequential access methods,	6
4	Concepts of High level,Assembly and Low level languages,	2
5	translator,assembler,Compiler,Interpreter,	2
6	Representing algorithms through flowchart and pseudocode.	2
7	Number System:Data represtnation,	6
8	Concepts of Redix and Representation of numbers in radix r with special case of r=2,8,10 and 16 with conversation from radix r1 to r2,r's	10
9	and (r-1)'s complement, binary addition,	10
10	Binary Subtraction,Represntation of Alphabets.	6

References:

1. Brian W.Kernighan and Dennis Ritchie,C Programming Langauge,Prantice Hall of India.
2. Yashwant Kanetkar,Let US C,BPB Publication.

28/3/2023

Faculty **HOD** **Dean Academics**

LECTURE -01 (UNIT-01)

Syllabus :-Fundamental of Computer

WHAT IS COMPUTER: -

Computer is an electronic device which accepts and processes the data by doing some mathematical and logical operations and gives us the desired output.

1.1 BASIC COMPUTER OPERATIONS

A computer basically performs five major operations or functions such as :

- Accepts data or instructions by way of input.
- Processes data as required by the user,
- Gives results in the form of output, and
- Controls all operations inside a computer.

1.2 ARCHITECTURE OF COMPUTER SYSTEM

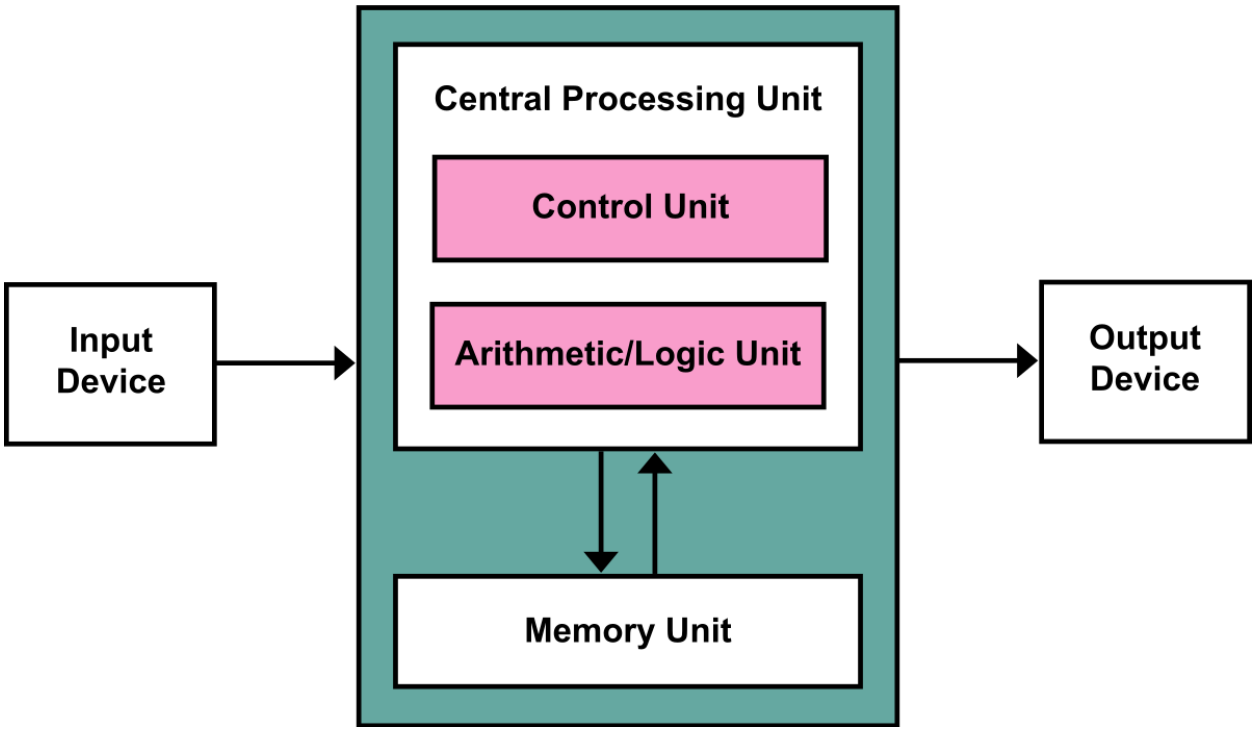


Fig 1.1: Architecture of Computer

In order to carry out its operations, a computer system is divided into three separate units. They are: 1) Arithmetic logical unit, 2) Control unit, and 3) Central processing unit. All these three units are known as functional units.

1.2.1Arithmetic Logical Unit (ALU)

The processing of the data and instructions are performed by Arithmetic Logical Unit. The major operations performed by the ALU are addition, subtraction, multiplication, division, logic and comparison. For processing, data is transferred from storage unit to ALU. After processing, the output is returned back to storage unit for further processing or for storing purpose.

1.2.2 Control Unit (CU)

The next component of computer is the Control Unit, which acts like the supervisor seeing that things are done in proper way. The control unit determines the sequence in which computer programs and instructions are to be executed. Activities like processing of programs stored in the main memory, interpretation of the instructions and issuing of signals for other units of the computer to execute them are carried out by CU. It coordinates the activities of computer's peripheral equipment which include input and output devices. Therefore, it is the manager of all the operations mentioned in the previous section.

1.2.3 Central Processing Unit (CPU)

The ALU and the CU of a computer system are jointly known as the central processing unit(CPU) . You may call CPU as the brain of any computer system. It is just like brain that takes all major decisions, makes all sorts of calculations and directs different parts of the computer functions by activating and controlling the operations. The CPU (Central Processing Unit) is the device that interprets and executes instructions. A computer system includes a computer, peripheral devices, and software.

REFERENCES:-

1. Brian W.Kernighan and Dennis Ritchie,C Programming Langauge,Prantice Hall of India.
2. Yashwant Kanetkar,Let US C,BPB Publication.

LECTURE -02 (UNIT-01)

1.3 CHARACTERISTICS OF COMPUTER

Let us now identify the major characteristics of a computer. These are:

1.3.1 Speed

As you know computer can work very fast. It takes only a fraction of a second for calculations that manually take hours to complete. It takes few minutes for the computer to process huge amount of data and give the result.

1.3.2Accuracy

The degree of accuracy of computer is very high and every calculation is performed with the same accuracy. The accuracy level is determined on the basis of design of the computer. The errors in computer are mainly due to human and inaccurate data.

1.3.3 Diligence

A computer is free from tiredness, lack of concentration, fatigue, etc. It can work for hours without any error.

1.3.4 Versatility

The computer is highly versatile. You can use it for a number of tasks simultaneously such as, for inventory management, preparation of electrical bills, preparation of pay cheques, etc. Similarly, in libraries computer can be used for various library housekeeping operations like acquisition, circulation, serial control, etc. and also by students for searching library books on the computer terminal.

1.3.5 Power of Remembering

Computer has the power of storing large amount of information or data. Any information can be stored and recalled whenever required for any numbers of years. It depends entirely upon you how much data you want to store in a computer and when to retrieve or delete stored data.

1.3.6 Dumb Machine with no IQ

Computer is a dumb machine and it cannot do any work without instructions from the user. It performs the instructions at a tremendous speed and with great accuracy as it has the power of logic. It is for you to decide what you want to do and in which sequence. So, a computer cannot take decision of its own as human beings can take.

1.3.7Storage

The computer has an in-built memory where it can store huge amount of data. You can also store data in secondary storage devices such as CDs, DVDs, and pen drives which can be kept outside the computer and can be carried to other computers.

1.4 GENERATION OF COMPUTERS

The history of computer development is in reference to different generation of computing devices. The first generation computers appeared in mid-1940s. The present day computer, however, has undergone rapid changes for the last seven decades. This period, during which the evolution of computer took place, can be divided into five distinct phases known as Generations of Computers that are being presented in the table given below:-

Generation Period Technology

First 1946-59 Based on vacuum tube technology Second 1957-64 Transistor based technology replaces vacuum tube

Third 1965-70 Integrated circuit (IC) technology developed

Fourth 1970-90 Microprocessors developed

Fifth 1990-till date Use of Bio-Chip technology

1.5TYPES OF COMPUTERS

Present day computers can be categorized as below:

1.5.1 Super Computer

Supercomputers are fastest computers and are very expensive. These are employed for specialized applications that require immense amounts of mathematical calculations. For example, weather forecasting requires a supercomputer. Other uses of supercomputers include animated graphics, fluid dynamic calculations, nuclear energy research, and petroleum exploration.

1.5.2 Mainframe Computer

It is a very large and expensive computer and is capable of supporting hundreds, or even thousands of users simultaneously. In the hierarchy that starts with a simple microprocessor (in watches, for example) at the bottom and moves to super computers at the top, mainframes are just below supercomputers. In some ways, mainframes are more powerful than supercomputers because they support simultaneous programs. But, supercomputers can execute a single program faster than a mainframe.

1.5.3. Laptop Computer: a portable computer complete with an integrated screen and keyboard. It is generally smaller in size than a desktop computer and largerthan a notebook computer.

1.5.4 Palmtop Computer/Digital Diary /Notebook /PDAs (Personal Digital Assistant): a hand-sized computer, Palmtop, does not have keyboard, but its screen serves both as an input and output device.

1.5.5Workstations

It is a terminal or desktop computer in a network. In this context, workstation is

just a generic term for a user's machine (client machine) in contrast to a "server" or "mainframe.

REFERENCES:-

1. Brian W.Kernighan and Dennis Ritchie,C Programming Langauge,Prantice Hall of India.
2. Yashwant Kanetkar,Let US C,BPB Publication.

LECTURE -03 (UNIT-01)

DATA STORAGE

Data storage is a common term for archiving data or information in a storage medium for use by a computer. It's one of the basic yet fundamental functions performed by a computer. It's like a hierarchy of comprehensive storage solution for fast access to computer resources. A computer stores data or information using several methods, which leads to different levels of data storage. Primary storage is the most common form of data storage which typically refers to the random access memory (RAM). It refers to the main storage of the computer because it holds data and applications that are currently in use by the computer. Then, there is secondary storage which refers to the external storage devices and other external media such as hard drive and optical media.

2.1 Primary Storage

Primary storage is commonly referred to as simply "primary memory" which is volatile in nature such as the RAM which is a primary memory and tends to lose data as soon as the computer reboots or loses power. It holds data or information that can be directly accessed by the central processing unit. RAM is stored in integrated circuits for immediate access with minimum or no delay. It's a high-speed data storage medium which is directly connected to the processing unit via the memory bus, allowing active programs to interact with the processor. Simple speaking, primary storage refers to internal storage devices that provide fast and efficient access to data or information. However, it stores data or applications for a short period of time while the computer is running.

2.2 Secondary Storage

Secondary storage is yet another ideal storage solution in the computer's memory hierarchy that is used to store data or information on the long term basis, more like permanently. Unlike primary storage, they are non-volatile memory or commonly referred to as external memory that are not directly accessed by the central processing unit. They are also called as auxiliary storage which can be both internal and external, plus beyond the primary storage. Because they are not directly accessed by the I/O channels, they are relatively slower than primary storage devices when it comes to data access. However, it's one of the most valuable assets of data storage hierarchy that is capable of storing applications and programs permanently. Unlike RAM, it's a long-term storage solution that expands the data storage capability.

Common example of secondary storage includes hard disk drives (HDD) which is the most common data storage device used to store and retrieve digital information. It's a high-capacity secondary storage device which also comes in internal storage mediums as internal hard drives. It's one of the most versatile mediums of data storage that uses magnetic storage to archive applications or data permanently. Other examples of secondary storage include optical media such as CDs and DVDs which are capable of storing any substantial amount of data; magnetic

tapes which are conventional methods of data storage used across corporate environments. However, secondary storage devices are quite slower than their primary counterparts, which make them relatively cheaper but equally efficient.

2.3 Difference between Primary and Secondary Storage

1. Storage : Data storage is the basic functionality of a computer which is divided into primary and secondary storage. primary storage refers to the main storage of the computer or main memory which is the random access memory or RAM.

   Secondary storage, on the other hand, refers to the external storage devices used to store data on a long-term basis.

2. Access of Primary Vs. Secondary Storage: Primary storage holds data or applications which can be directly accessed by the processing unit with minimum or no delay.

   On the contrary, secondary storage is used to store and retrieve data permanently with no delay.

3. Nature of Primary Vs. Secondary Storage :Primary storage is a volatile memory which means data is lost as soon as the device loses power and it cannot be retained. Primary storage is commonly referred to as primary memory such as the RAM.

   Secondary storage, commonly known as secondary memory, is a non-volatile memory which is able to retain data even if the device loses power.

4. Device used for Primary Vs. Secondary Storage: RAM is the most common primary storage device which also goes by main memory and is used to store data machine code currently in use. Instructions can be retrieved from the RAM whenever required. It provides fast data access with no delay.

   Secondary storage refers to external storage devices such as optical media (CDs and DVDs), hard disk drives (HDD), floppy disks, USB flash drives, etc.

5. Speed of Primary Vs. Secondary Storage: As programs and applications are stored in main memory, primary storage provides fast and efficient access to the CPU.

On the contrary, secondary storage is more of a long-term storage solution with substantial data storage capacity which makes them relatively slower than their primary counterparts

What is a programming language?

A programming language defines a set of instructions that are compiled together to perform a specific task by the CPU (Central Processing Unit). The programming language mainly refers to high-level languages such as C, C++, Pascal, Ada, COBOL, etc.Each programming language contains a unique set of keywords and syntax, which are used to create a set of instructions. Thousands of programming languages have been developed till now, but each language has its specific purpose. These languages vary in the level of abstraction they provide from the hardware. Some programming languages provide less or no abstraction while some provide higher abstraction. Based on the levels of abstraction, they can be classified into two categories:

- Low-level language
- High-level language

The image which is given below describes the abstraction level from hardware. As we can observe from the below image that the machine language provides no abstraction, assembly language provides less abstraction whereas high-level language provides a higher level of abstraction.

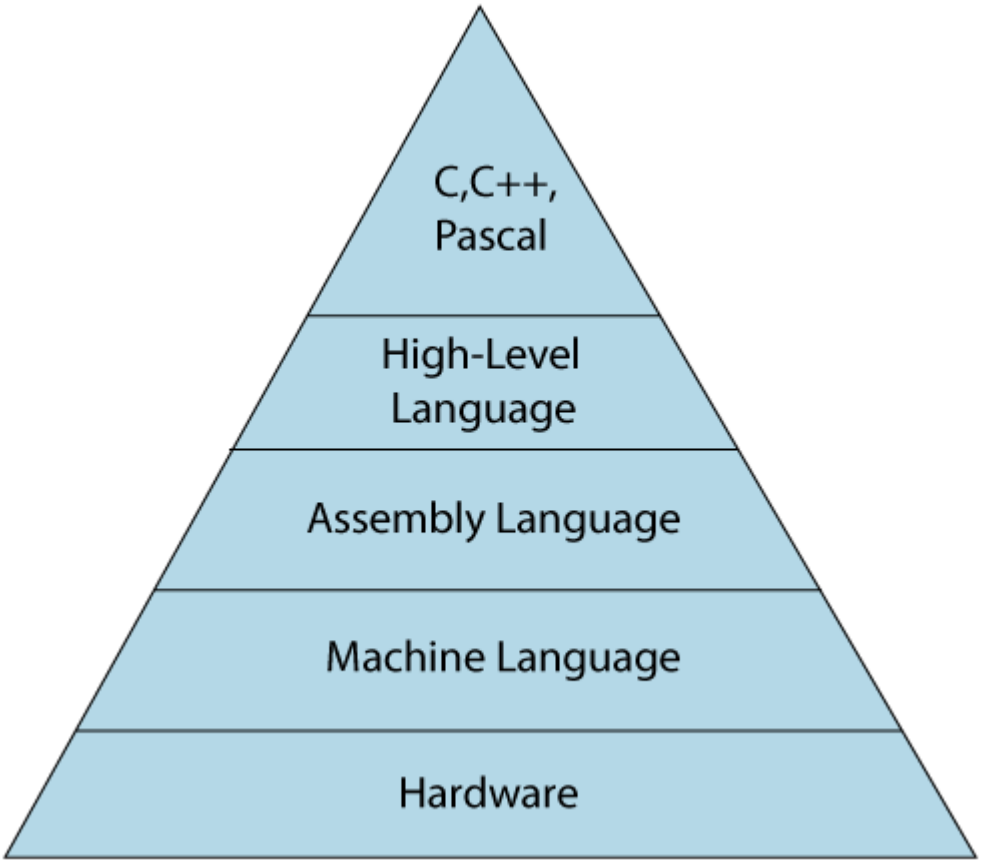


Fig: 1.2 Level of Lenguages

Low-level language

The low-level language is a programming language that provides no abstraction from the hardware, and it is represented in 0 or 1 forms, which are the machine instructions. The languages that come under this category are the Machine level language and Assembly language.

Machine-level language

The machine-level language is a language that consists of a set of instructions that are in the binary form 0 or 1. As we know that computers can understand only machine instructions, which are in binary digits, i.e., 0 and 1, so the instructions given to the computer can be only in binary codes. Creating a program in a machine-level language is a very difficult task as it is not easy for the programmers to write the program in machine instructions. It is error-prone as it is not easy to understand, and its maintenance is also very high. A machine-level language is not portable as each computer has its machine instructions, so if we write a program in one computer will no longer be valid in another computer.

The different processor architectures use different machine codes, for example, a PowerPC processor contains RISC architecture, which requires different code than intel x86 processor, which has a CISC architecture.

Assembly Language:

The assembly language contains some human-readable commands such as mov, add, sub, etc. The problems which we were facing in machine-level language are reduced to some extent by using an extended form of machine-level language known as assembly language. Since assembly language instructions are written in English words like mov, add, sub, so it is easier to write and understand.

As we know that computers can only understand the machine-level instructions, so we require a translator that converts the assembly code into machine code. The translator used for translating the code is known as an assembler.

The assembly language code is not portable because the data is stored in computer registers, and the computer has to know the different sets of registers.

Differences between Machine-Level language and Assembly language

The following are the differences between machine-level language and assembly language:

Machine-level language	Assembly language
The machine-level language comes at the	The assembly language comes above the

lowest level in the hierarchy, so it has zero abstraction level from the hardware.	machine language means that it has less abstraction level from the hardware.
It cannot be easily understood by humans.	It is easy to read, write, and maintain.
The machine-level language is written in binary digits, i.e., 0 and 1.	The assembly language is written in simple English language, so it is easily understandable by the users.
It does not require any translator as the machine code is directly executed by the computer.	In assembly language, the assembler is used to convert the assembly code into machine code.
It is a first-generation programming language.	It is a second-generation programming language.

High-Level Language

The high-level language is a programming language that allows a programmer to write the programs which are independent of a particular type of computer. The high-level languages are considered as high-level because they are closer to human languages than machine-level languages.

When writing a program in a high-level language, then the whole attention needs to be paid to the logic of the problem.

A compiler is required to translate a high-level language into a low-level language.

Advantages of a high-level language

- The high-level language is easy to read, write, and maintain as it is written in English like words.
- The high-level languages are designed to overcome the limitation of low-level language, i.e., portability. The high-level language is portable; i.e., these languages are machine-independent.

  Differences between Low-Level language and High-Level language

  The following are the differences between low-level language and high-level language:

Low-level language	High-level language
It is a machine-friendly language, i.e., the computer understands the machine language, which is represented in 0 or 1.	It is a user-friendly language as this language is written in simple English words, which can be easily understood by humans.
The low-level language takes more time to execute.	It executes at a faster pace.
It requires the assembler to convert the assembly code into machine code.	It requires the compiler to convert the high-level language instructions into machine code.

The machine code cannot run on all machines, so it is not a portable language.	The high-level code can run all the platforms, so it is a portable language.

REFERENCES:-

1. Brian W.Kernighan and Dennis Ritchie,C Programming Langauge,Prantice Hall of India.
2. Yashwant Kanetkar,Let US C,BPB Publication.

LECTURE -04 (UNIT-01)

HARDWARE AND SOFTWARE

4.1Hardware

Hardware refers to the physical equipment used for the input, processing,output and storage activities of a computer system. It consists of mechanical and electronic devices, which we are able to see and touch easily. Some of them are central processing unit (CPU), primary storage devices, secondary storage devices, input and output unit and communication devices. These are explained below:-

- Central processing unit (CPU): It manipulates the data and controls the tasks performed by the other components.
- Primary storage: It stores temporarily data and program instructions during the processing.
- Primary memory (main memory): These are RAM (Random Access Memory/Read-Write Memory), and ROM (Read-only-memory).
- Secondary storage: These store data and programs for future use. These are Hard Disk (Local Disk) and External Hard Disc, Optical Disks,(CDR, CD-RW, DVD-R, DVD-RW), Pen Drive, Memory Cards, etc

4.2 Software : is a broad term for the programs running on hardware. Familiar kinds of software are operating systems, which provide overall control for computer hardware, and applications, which are optional programs used for a particular job. Software resides on disks and is brought into memory when it is needed.

4.3 Firmware: Often a distinction is drawn between software and firmware. Firmware is software that is semi-permanently placed in hardware. It does not disappear when hardware is powered off, and is often changed by special installation processes or with administration tools. The memory firmware uses is very fast — making it ideal for controlling hardware where performance is important. For example, NETGEAR routers filter using firmware, which tends to make them faster than a PC or Macintosh performing a similar function.

A driver is software and/or firmware that controls hardware. Often it connects an operating system with specific hardware devices. For example, there are drivers for every card and disk in your computer. Each driver is written for a specific operating system — for example Windows XP or Macintosh OS X. Therefore, to use a card in your computer, you must use a driver that matches the device and also your operating system. Drivers can be enhanced, for example, when new operating systems come out. Eventually hardware becomes so old it is no longer economical or practical to produce new drivers for it.

Sometimes the words software, firmware and driver are used interchangeably, so don't be thrown off if somebody uses the word "software" when you expected to hear "driver", or vice versa.

A utility is software used for the limited purpose of changing the overall behaviour of hardware or other software. (For example configuring your browser to accept cookies.) Utilities tend to be used once or twice at most. On a typical computer or router, there will be utilities users never touch at all. If a utility is not used, default values are used, instead.

REFERENCES:-

1. Brian W.Kernighan and Dennis Ritchie,C Programming Langauge,Prantice Hall of India.
2. Yashwant Kanetkar,Let US C,BPB Publication.

LECTURE -05 (UNIT-01)

Translators, compilers, interpreters and assemblers: are all software programming tools that convert code into another type of code, but each term has specific meaning. All of the above work in some way towards getting a high-level programming language translated into machine code that the central processing unit (CPU) can understand. Examples of CPUs include those made by Intel (e.g., x86), AMD (e.g., Athlon APU), NXP (e.g., PowerPC), and many others. It's important to note that all translators, compilers, interpreters and assemblers are programs themselves.

5.1Translators

The most general term for a software code converting tool is "translator." A translator, in software programming terms, is a generic term that could refer to a compiler, assembler, or interpreter; anything that converts higher level code into another high-level code (e.g., Basic, C++, Fortran, Java) or lower-level (i.e., a language that the processor can understand), such as assembly language or machine code. If you don't know what the tool actually does other than that it accomplishes some level of code conversion to a specific target language, then you can safely call it a translator.

5.2Compilers

Compilers convert high-level language code to machine (object) code in one session. Compilers can take a while, because they have to translate high-level code to lower-level machine language all at once and then save the executable object code to memory. A compiler creates machine code that runs on a processor with a specific Instruction Set Architecture (ISA), which is processor-dependent. For example, you cannot compile code for an x86 and run it on a MIPS architecture without a special compiler. Compilers are also platform-dependent. That is, a compiler can convert C++, for example, to machine code that's targeted at a platform that is running the Linux OS. A cross-compiler, however, can generate code for a platform other than the one it runs on itself.

5.3Interpreters

Another way to get code to run on your processor is to use an interpreter, which is not the same as a compiler. An interpreter translates code like a compiler but reads the code and immediately executes on that code, and therefore is initially faster than a compiler. Thus, interpreters are often used in software development tools as debugging tools, as they can execute a single in of code at a time. Compilers translate code all at once and the processor then executes upon the machine language that the compiler produced. If changes are made to the code after compilation, the changed code will need to be compiled and added to the compiled code (or perhaps the entire program will need to be re-compiled.) But an interpreter, although skipping the step of compilation of the entire program to start, is much slower to execute than the same program that's been completely compiled**.**

5.4Assemblers

An assembler translates a program written in assembly language into machine language and is effectively a compiler for the assembly language, but can also be used interactively like an interpreter. Assembly language is a low-level programming language. Low-level programming languages are less like human language in that they are more difficult to understand at a glance; you have to study assembly code carefully in order to follow the intent of execution and in most cases, assembly code has many more lines of code to represent the same functions being executed as a higher-level language. An assembler converts assembly language code into machine code (also known as object code), an even lower-level language that the processor can directly understand.

Assembly language code is more often used with 8-bit processors and becomes increasingly unwieldy as the processor's instruction set path becomes wider (e.g., 16-bit, 32-bit, and 64-bit). It is not impossible for people to read machine code, the strings of ones and zeros that digital devices (including processors) use to communicate, but it's likely only read by people in cases of computer forensics or brute-force hacking. Assembly language is the next level up from machine code, and is quite useful in extreme cases of debugging code to determine exactly what's going on in a problematic execution, for instance. Sometimes compilers will "optimize" code in unforeseen ways that affect outcomes to the bafflement of the developer or programmer such that it's necessary to carefully follow the step-by-step action of the processor in assembly code, much like a hunter tracking prey or a detective following clues.

REFERENCES:-

1. Brian W.Kernighan and Dennis Ritchie,C Programming Langauge,Prantice Hall of India.
2. Yashwant Kanetkar,Let US C,BPB Publication.

LECTURE -06 (UNIT-01)

ALGORITHM

6.1 Algorithm: Set of step-by-step instructions that perform a specific task or operation Natural‖ language NOT programming language

6.2 Pseudocode: Set of instructions that mimic programming language instructions

6.3 Flowchart: Visual program design tool —Semantic‖ symbols describe operations to be performed.

Definitions: A flowchart is a schematic representation of an algorithm or a stepwise process, showing the steps as boxes of various kinds, and their order by connecting these with arrows. Flowcharts are used in designing or documenting a process or program.

An object file is a computer file containing object code, that is, machine code output of an assembler or compiler. The object code is usually relocatable, and not usually directly executable. There are various formats for object files, and the same machine code can be packaged in different object file formats. An object file may also work like a shared library. A computer programmer generates object code with a compiler or assembler. For example, under Linux, the GNU Compiler Collection compiler will generate files with a .o extension which use the ELF format. Compilation on Windows generates files with a .obj extension which use the COFF format. A linker is then used to combine the object code into one executable program or library pulling in precompiled system libraries as needed.

Source code is generally understood to mean programming statements that are created by a programmer with a text editor or a visual programming tool and then saved in a file. Object code generally refers to the output, a compiled file, which is produced when the Source Code is compiled with a C compiler.

REFERENCES:-

1. Brian W.Kernighan and Dennis Ritchie,C Programming Langauge,Prantice Hall of India.
2. Yashwant Kanetkar,Let US C,BPB Publication.

LECTURE -07 (UNIT-01)

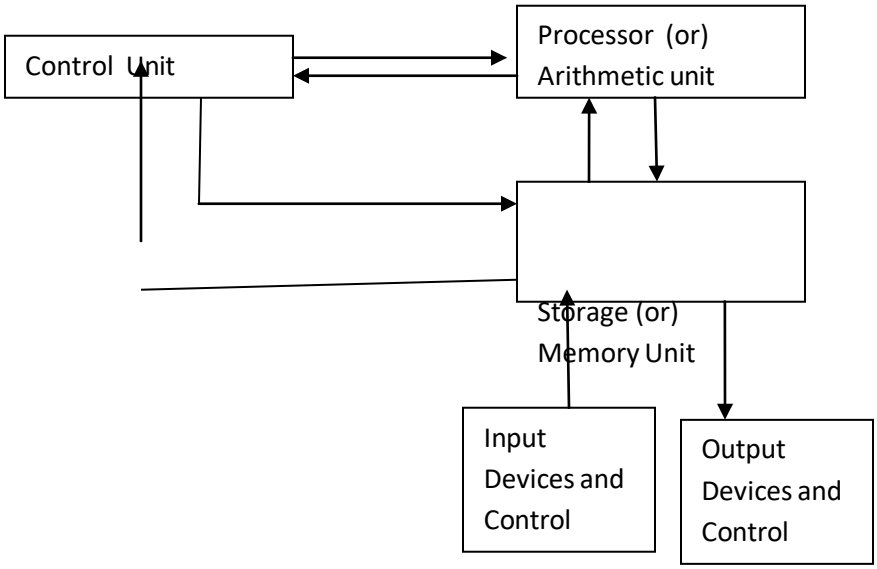
Number System

A number system relates quantities and symbols. In digital system how information is represented is key and there are different radices, i.e. number base that a numbering system can use.

1. Digital computer

Any class of devices capable of solving problems by processing information in discrete form. It operates on data, including letters and symbols that are expressed in binary form i.e using only two digits 0 and 1.

The block diagram of digital computer is given below:



The memory unit stores programs as well as input, output and intermediate data. The processor unit performs arithmetic and other data processing tasks as specified by the

2. Binary numbers

Numbers that contain only two digit 0 and 1 are called Binary Numbers. Each 0 or 1 is called a Bit, from binary digit. A binary number of 4 bits is called a Nibble. A binary number of 8 bits is called a Byte. A binary number of 16 bits is called a Word on some systems, on others a 32-bit number is called a Word while a 16-bit number is called a Halfword.

Using 2 bit 0 and 1 to form

a binary number of 1 bit, numbers are 0 and 1

a binary number of 2 bit, numbers are 00, 01, 10, 11

a binary number of 3 bit, such numbers are 000, 001, 010, 011, 100, 101, 110, 111
a binary number of 4 bit, such numbers are 0000, 0001, 0010, 0011, 0100, 0101, 0110, 0111, 1000,
1001, 1010, 1011, 1100,1101,1110,1111

Therefore , using n bits there are 2n binary numbers of n bits

Each digit in a binary number has a value or weight. The LSB has a value of 1. The second from the right has a value of 2, the next 4 , etc.,

16	*8*	*4*	*2*	*1*
2^4	2^3	2^2	2^1	2^0

The binary equivalent for some decimal numbers are given below.

Decimal	0	1	2	3	4	5	6	7	8	9	10	11
Binary	0	1	10	11	100	101	110	111	1000	1001	1010	1011

Number Base Conversions

Conversion of decimal number to any number system

Step 1 convert the integer part by doing successive division using the radix of asked number systems. Step 2 convert the fractional part by doing successive multiplication using radix of asked number system

Conversion of decimal to binary :number system The radix of asked number system is 2

Convert 87_{10} to $(\ )_2$

2	87	→ 1
2	43	→ 1
2	21	→ 1
2	10	→ 0
2	5	→ 1
2	2	→ 0
	1	

$(1010111)_2$

Convert $(14.625)_{10}$ decimal number to binary number

2	14
2	7-0
2	3-1
	1-1

MSB

1st Multiplication Iteration Multiply 0.625 by 2

$(1110)_2$

0.625 x 2 = 1.25(Product) Fractional part=0.25 Carry=1 (MSB)

2nd Multiplication Iteration Multiply 0.25 by 2

0.25 x 2 = 0.50(Product) Fractional part = 0.50 Carry = 0

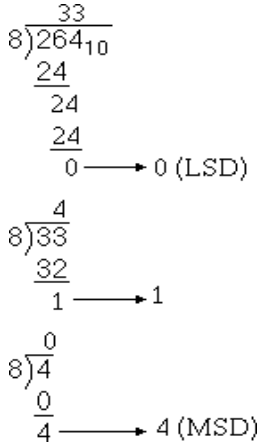
3rd Multiplication Iteration Multiply 0.50 by 2

0.50 x 2 = 1.00(Product) Fractional part = 1.00 Carry = 1 (LSB)

$(101)_2$

The binary number of $(16.625)_{10}$ is $(1110.101)_2$

Conversion of decimal to octal number system The radix of asked number system is 8

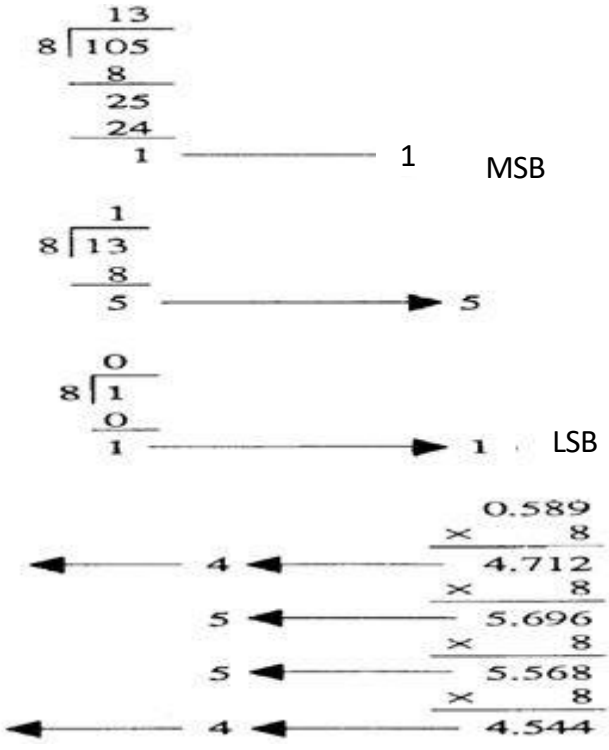


Convert $(264)_{10}$ decimal number to octal number

$(410)_8$

The octal number of $(264)_{10}$ is $(410)_8$

Convert $(105.589)_{10}$ decimal number to octal number



(151)

MSB

LSB

The octal number of $(105.589)_{10}$ is $(151.4554)_8$

3.4 Conversion of decimal to Hexadecimal number system The radix of asked number system is 16

Convert $(1693)_{10}$ decimal number to Hexadecimal number 1693/16 = 105 Reminder (13) D (LSB)

105/16 = 6 Reminder 9

6/16 =0 Reminder 6 (MSB)

(0.4554)

(1693)10 (69D)16

Convert $(1693.0628)_{10}$ decimal fraction to hexadecimal fraction $(?)_{16}$ 1693/16 = 105 Reminder (13) D (LSB)

105/16 = 6 Reminder 9

6/16 = 0 Reminder 6 (MSB) (69D)

Multiply 0.0628 by 16

0.0628 x 16 = 1.0048(Product) Fractional part=0.0048 Carry=1 (MSB) Multiply 0.0048 by 16

0.0048 x 16 = 0.0768(Product) Fractional part = 0.0768 Carry = 0

Multiply 0.0768 by 16

0.0768 x 16 = 1.2288(Product) Fractional part = 0.2288 Carry = 1

Multiply 0.2288 by 16

0.2288 x 16 = 3.6608(Product) Fractional part = 0.6608 Carry = 3 (LSB)

(.1013)

$(1693.0628)_{10} = (69D.1013)_{16}$

REFERENCES:-

1. Brian W.Kernighan and Dennis Ritchie,C Programming Langauge,Prantice Hall of India.
2. Yashwant Kanetkar,Let US C,BPB Publication.

LECTURE -08 (UNIT-01)

Conversion of any number system to decimal number system

In general the numbers can be represented as

$N = A_{n-1} r_{n-1} + = A_{n-2} r_{n-2} + \ldots\ldots\ldots + A_1 r^1 + A_0 r^0 + A_{-1} r^{-1} + A_{-2} r^{-2} + \ldots\ldots\ldots$

Where n= number in decimal A= digit

r= radix of number system

n= The number of digits in the integer portion of number m= the number of digits in the fractional portion of number

Conversion of binary to decimal number system Convert $(101.101)_2 = (?)_{10}$

101.101

$= 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 . \ 1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3}$

$= 1 \times 4 + 0 \times 2 + 1 \times 1 . \ 1 \times (1 / 2) + 0 \times (1 / 4) + 1 \times (1 / 8)$

$= 4 + 0 + 1 . (1 / 2) + 0 + (1 / 8)$

$= 5 + 0.5 + 0.125$

$= 5 . 625$

Therefore $(1\ 0\ 1 . 1\ 0\ 1)_2 = (5.625)_{10}$

Conversion of octal to decimal number system Convert $(128)_8 = (?)_{10}$

$123_8 = 1*8^2 + 2*8^1 + 3*8^0 = 64 + 16 + 3 = 73$

the decimal equivalent of the number 123_8 is 73_{10} Convert $(2\ 1.\ 2\ 1)_8 = (?)_{10}$

2 1. 2 1

$= 2 \times 8^1 + 1 \times 8^0 . \ 2 \times 8^{-1} + 1 \times 8^{-2}$

$= 2 \times 8 + 1 \times 1. \ 2 \times (1 / 8) + 1 \times (1 / 64)$

$= 16 + 1 . (0. 2\ 5) + (0. 0\ 1\ 5\ 6\ 2\ 5)$

$= 17 + 0. 265625$

= 17. 265625

Therefore $(2\ 1.\ 2\ 1)_8 = (1\ 7.\ 2\ 6\ 5\ 6\ 2\ 5)_{10}$

Conversion of hexadecimal to decimal number system

Convert $(E\ F.\ B\ 1)_{16} = (?)_{10}$

$= E \times 16^1 + F \times 16^0.\ B \times 16^{-1} + 1 \times 16^{-2}$

= 14 x 16 + 15 x 1 . 11 x (1 / 16) + 1 x (1 / 256)

= 224 + 15 + (0. 6 8 7 5) + (0. 0 0 3 9 0 6 2 5)

= 239 + 0. 6914

= 239. 691406

Therefore $(E\ F.\ B\ 1)_{16} = (2\ 3\ 9.\ 6\ 9\ 1\ 4\ 0\ 6)_{10}$ Convert (0.9D9)16= (?)10

= 0 x 160. 9 x 16-1 + D x 16-2 + 9 x 16-3

= 0 x 1. 9 x (1 / 16) + 13 x (1 / 256) + 9 x (1 / 4096)

= 0 . (0. 5625) + (0. 050781) + (0. 0021972)

= 0. (0. 6154782)

0. 6154782

Conversion of binary to octal number system Convert $(101101001)_2$ to ( )8

Divide the binary into group of three digits from LSB we will find the following pattern 101|101|001 Now writing the equivalent decimal number of each group we get 5 | 5 | 1 So the equivalent octal number is 551_8

Convert 11001100.101 to ( )8 011|001|100. |101|

3 1 4 . 5

So the equivalent octal number is 314.5

Conversion of binary to hexadecimal number system Convert 111100010 to ( )16

Divide the binary into group of four digits from LSB 0001|1110|0010

Now writing the equivalent hexadecimal number of each group

1|E|2

So the equivalent Hexa decimal number is $1E2_{16}$ Convert 11000011001.101 to ( )$_{16}$

0110|0001|1001|.1010|

6 1 9 . A

So the equivalent Hexa decimal number is $619.A_{16}$

Conversion of octal number system to hexa decimal number system Convert $(25)_8$ to ( )$_{16}$

First convert octal to binary

The binary equivalent of 25 is 010101

Divide the binary into group of four digits from LSB 0001|0101

1 5

So the equivalent Hexa decimal number is 15_{16}

Conversion of hexa decimal number system to octal number system Convert $(1A.2B)_{16}$ to ( )$_8$

First convert hexadecimal to binary

The binary equivalent of 1A.2B is 00011010.00101011 Divide the binary into group of Three digits 011|010|.|001|010|110

3 2 . 1 2 6

so the equivalent octal number is 32.126_8

COMPLEMENTS

In digital computers to simplify the subtraction operation and for logical manipulation complements are used . There are two types of complements for each radix system the radix complement and diminished radix complement. The first is referred to as the r's complement and the second as the (r-1)'s complement.

r's Complement

Given a positive number N in base r with an integer part of n digits, the r's complement of N is

defined as r^n-N if N≠0 and 0 if N=0

(r-1)'s Complement

Given a positive number N in base r with an integer part of n digits and a fraction part of m digits, the (r-1)'s complement of N is defined as r^n-r^{-m}-N

Subtraction with r's complement

- The direct method of subtraction uses the borrow concept
- When subtraction is implemented by means of digital components, this method is found to be less efficient. So, instead the following procedure can be followed.

The subtraction of two positive numbers (M-N), both of base r, may be done as follows.

(1) Add the minuend M to the r's complement of the subtrahend N.
(2) Inspect the result obtained in step 1 for an end carry.
If an end-carry occurs, discard it.
If an end-carry does not occurs, take the r's complement of the number obtained in step 1 and place a negative sign in front.

Subtraction with (r-1)'s Complement

- The procedure for subtraction with (r-1)'s complement is same as r's complement except for end-around carry.
- The subtraction of M-N, both positive numbers in base r, may be calculated in the following manner.
    1. Add the minuend M to the (r-1)'s complement of the subtrahend N.
    2. Inspect the result obtained in step 1 for an end carry.
        - If an end-carry occurs, add 1 to the least significant digit (end-around carry)
        - If an end-carry does not occur, take the (r-1)'s complement of the number obtained in step 1 and place a negative sign in front.

It is classified into four types they are 1's complement , 2's complement , 9's complement and 10's complement.

1's complement representation: The 1's complement of a binary number is the number that results when we change all 1's to zeros and the zeros to ones.

2's complement representation:

The 2's complement is the binary number that results when we add 1 to the 1's complement.

Problems related to 1's complement and 2's complement :

1. *Express the following numbers in sign magnitude 1's and 2's complement :*
i) –56 ii)107

Solution : i) – 56

	56 =	0 1 1 1 0 0 0	
	– 56 =	1 0 0 0 1 1 1	1's Complement
		+ 1	
	=	1 0 0 1 0 0 0	2's Complement
ii) 107	107 =	0 1 1 0 1 0 1 1	
	–107 =	1 0 0 1 0 1 0 0	1's Complement
		+ 1	
	=	1 0 0 1 0 1 0 1	2's Complement.

2. *Find 2's complement of* $(1\ 0\ 0\ 1)_2$

Solution :

1 0 0 1	number
0 1 1 0	1's complement
+ 1	
0 1 1 1	2's complement

3. *Find 2's complement of* $(1\ 0\ 1\ 0\ \ 0\ 0\ 1\ 1)_2$

Solution :

1 0 1 0 0 0 1 1	number
0 1 0 1 1 1 0 0	1's complement
+ 1	
0 1 0 1 1 1 0 1	2's complement

1's complement subtraction

Subtraction of binary numbers can be accomplished by the direct method by using the 1's complement method, which allows to perform subtraction using only addition . for subtraction of two numbers we have two cases.

1. Subtraction of smaller number from larger number and
2. Subtraction of larger number from smaller number.

1's complement Subtraction of smaller number from larger number

Method:

1. Determine the 1's complement of the smaller number.

2. Add the 1's complement to the larger number.
3. Remove the carry and add it to the result. This is called end -around carry.
4. *Subtract* 101011_2 *from* 111001_2 *using the 1's complement method.*

Solution :

```
  1 1 1 0 0 1
+ 0 1 0 1 0 0   1's complement of 1 0 1 0 1 1
-------------
① 0 0 1 1 0 1
  └──→+   1    Add end-around carry
-------------
  0 0 1 1 1 0   Final answer
```

1's complement Subtraction of larger number from smaller number

Method:

1. Determine the 1's complement of the larger number.
2. Add the 1's complement to the smaller number.
3. Answer is in 1's complement form. To get the answer in true form take the 1's complement and assign negative sign to the answer.

5. *Subtract* 111001_2 *from* 101011_2 *using the 1's complement method.*

Solution :

```
  1 0 1 0 1 1
+ 0 0 0 1 1 0   1's complement of 1 1 1 0 0 1
-------------
  1 1 0 0 0 1   Answer in 1's complement form
- 0 0 1 1 1 0   Answer in true form
```

Advantages of 1's complement subtraction:

1. The 1's complement subtraction can be accomplished with an binary adder. Therefore , this method is useful in arithmetic logic circuits.
2. The 1's complement of a number is easily obtained by inverting each bit in the number.

2's complement Subtraction:

Like 1's complement subtraction, in 2's complement subtraction, the subtraction is accomplished

by only addition.

2's complement Subtraction of smaller number from larger number

Method

1. Determine the 2's complement of the smaller number.
2. Add the 2's complement to the larger number.
3. Discard the carry.

6. *Subtract* 101011_2 *from* 111001_2 *using the 2's complement method.*

Solution :

```
  1 1 1 0 0 1
+ 0 1 0 1 0 1     2's complement of 1 0 1 0 1 1
-----------
  0 0 1 1 1 0
  0 0 1 1 1 0     Final answer
```

2's complement Subtraction of larger number from smaller number

Method:

1. Determine the 2's complement of the larger number.
2. Add the 2's complement to the smaller number.
3. Answer is in 2's complement form. To get the answer in true form take the 2's complement and assign negative sign to the answer.

7. *Subtract* 111001_2 *from* 101011_2 *using 2's complement method.*

Solution :

```
  1 0 1 0 1 1
+ 0 0 0 1 1 1     2's complement of 1 1 1 0 0 1
-----------
  1 1 0 0 1 0     Answer in 2's complement form
- 0 0 1 1 1 0     Answer in true form
```

9's complement and 10's complement

Before knowing about 9's complement and 10's complement we should know why they are used and why their concept came into existence. Addition of signed BCD numbers can be performed by using 9's and 10's complement. The complements are used to make the arithmetic operations in digital system easier. Various topics and related problems we going to see here are

1. 9s complement
2. 10s complement
3. 9s complement subtraction
4. 10s complement subtraction

Now first of all let us know what 9's complement is and how it is done. To obtain the 9,s complement of any number we have to subtract the number with (10n - 1) where n = number of digits in the number, or in a simpler manner we have to divide each digit of the given decimal number with 9. The table 1. Will explain the 9's complement more easily.

Decimal digit	9s complement
0	9
1	8
2	7
3	6
4	5
5	4
6	3
7	2
8	1
9	0

Table 1. 9's complement equivalent for decimal numbers

Now coming to 10's complement, it is relatively easy to find out the 10's complement after finding out the 9,s complement of that number. We have to add 1 with the 9,s complement of any number to obtain the desired 10's complement of that number. Or if we want to find out the 10's complement directly, we can do it by following the formula, (10^n - number), where n = number of digits in the number. An example is given below to illustrate the concept of obtaining 10's complement.

A decimal number 456, find 9's complement and 10's complement of this number

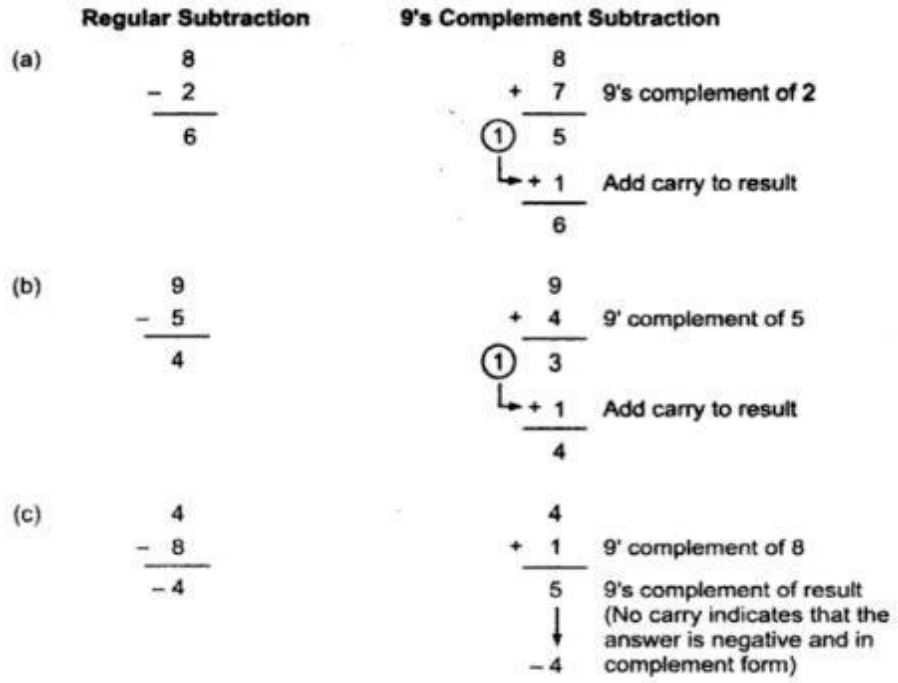
```
   999
(-) 456
-------
   543
```

10's complement of that no. is

```
543
(+)1
---
544
```

In 9's complement subtraction when 9's complement of smaller number number is added to the larger number carry is generated. It is necessary to add this carry to the result. (this is called an end around carry).when larger number is subtracted from the smaller number, there is no carry, and the result is in 9's compliment form and negative. This is explained with following examples.

Subtraction using 9's complements:

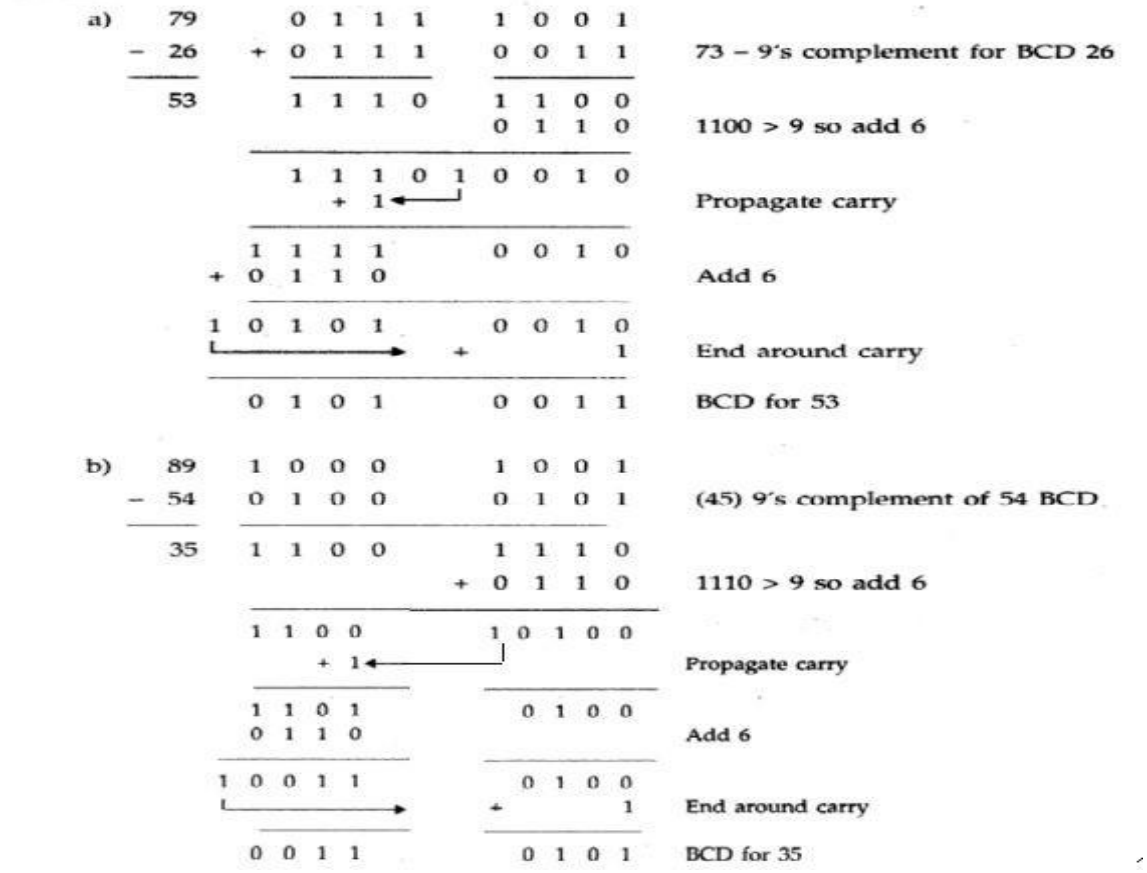


Steps for 9's complement BCD subtraction

1. Find the 9's complement of a negative number.
2. Add two numbers using BCD addition
3. If carry is generated add carry to the result otherwise find the 9's complement of the result.

9. *Perform each of the following decimal subtractions in 8-4-2-1 BCD using 9's complement method.* a) 79 − 26 b) 89 − 54

Solution :



Subtraction using 10's complements:

The 10's complement of the decimal is equal to 9's complement plus 1. The 10's complement can be used to perform subtraction by adding the minuend to the 10's complement of the subtrahend and dropping the carry. This is explained with following examples.

	Regular Subtraction	10's Complement Subtraction	
(a)	8	8	
	− 2	+ 8	10's complement of 2
	6	✗6	Drop carry
(b)	9	9	
	− 5	+ 5	10's complement of 5
	4	✗4	Drop carry
(c)	4	4	
	− 8	+ 2	10's complement of 8
	−4	6	10's complement of result
		↓	(No carry indicates that the answer is negative and in the 10's complement form)
		−4	

Steps for 10's complement BCD subtraction

1. Find the 10's complement of a negative number.
2. Add two numbers using BCD addition
3. If carry is not generated find the 10's complement of the result.

REFERENCES:-

1. Brian W.Kernighan and Dennis Ritchie,C Programming Langauge,Prantice Hall of India.
2. Yashwant Kanetkar,Let US C,BPB Publication.

LECTURE -09 (UNIT-01)

BINARY ARITHMETIC

BINARY ADDITION

The binary addition table is as follows:

A+B	SUM	CARRY
0+0	0	0
0+1	1	0
1+0	1	0
1+1	0	1

Illustration 1:

Add $(1010)_2$ and

1010 $(0011)_2$

(Augend)

0011 (Addend)

1101 (sum)

The addition manipulated above as follows.

Step 1: The least significant bits are added, i.e. 0+1 =1 with a carry of 0

Step 2: The carry in the previous is added to the next higher significant bits, i.e. 0+1+1= 0 with a carry 1. Step 3: The carry in the previous is added to the next higher significant bits, i.e. 1+0+0 =1 with a carry 0. Step 4: The preceding carry is added to the most significant bit i.e. 0+1+0 = 1 with a carry 0.

Thus the sum is 1101.

BINARY SUBTRACTION

The binary subtraction table is as follows:

A-B	DIFFERENCE	BORROW
0-0	0	0
0-1	1	1

1-0	1	0
1-1	0	0

Illustration 1:

Subtract $(0101)_2$ from $(1011)_2$

1011 (Minuend)
0101 (Subtrahend)

0110 (Difference)

The steps are described below

Step1: the LSB in the first column are 1 and 1. Hence, the difference is 1 - 1 = 0
Step2: The column, the subtraction is performed as 1 – 0 = 1
Step3: In the third column, the difference is given by 0 – 1 =1
Step 4: In the fourth column (MSB), the difference is given by 0 – 0 = 0 since 1 is borrowed for third column.

BINARY MULTIPLICATION

The binary multiplication table is as follows:

A *B	PRODUCT
0 * 0	0
0 * 1	0
1 * 0	0
1 * 1	1

- Binary multiplication uses add and shift process
- Binary multiplication is similar to decimal multiplication.

Illustration 1:

Multiplicand * Multiplier
10110.1x01001.1

```
                101101
           101101
            000000
           000000
           101101
           000000
-------------------------

     011010101.11
-------------------------
```

Partial Product

(Final product)

The steps are described below

Step 1: The LSB of the multiplier is taken. If multiplier bit is 1, the multiplicand is copied as such and if the multiplier bit is 0 zero is placed in all the bit positions.

Step 2: The next higher significant bit of the multiplier is taken and, the partial product is written with the shift to the left, as in step 1.

Step 3: step 2 is repeated for all other higher significant bits.

Step 4: The partial product terms are added which gives the actual product of multiplier and the multiplicand.

BINARY DIVISION:

The binary division table is as follows:

A÷B	Result
0÷0	Not allowed
0÷1	0
1÷0	Not allowed
1÷1	1

- Binary division uses subtract and shift process
- Binary division is similar to decimal division.
- Division by 0 is meaningless.

Illustration 1:

Dividend ÷ Divisor
11011.1÷ 101
101.1
(QUOTI
ENT) DIVISOR 101 √~~11011.1~~
(DIVIDE
ND)
101

111
101

101
101

0

BINARY CODES

Binary codes are codes which are represented in binary system with modification from the original one. The group of symbols is called as a code. The digital data is represented, stored and transmitted as group of binary bits. This group is also called as binary code. The binary code is represented by the number as well as alphanumeric letter.

Advantages of Binary Code

Following is the list of advantages that binary code offers.

1. Binary codes are suitable for the computer applications.
2. Binary codes are suitable for the digital communications.
3. Binary codes make the analysis and designing of digital circuits if we use the binary codes.
4. Since only 0 and 1 are being used, implementation becomes easy.

*ification of binary codes:*The codes are broadly categorized into following four categories.

- Weighted Codes
- Non-Weighted Codes
- Binary Coded Decimal Code
- Alphanumeric Codes

- Error Codes

  ted codes: Weighted binary codes are those binary codes which obey the positional weight principle. Each position of the number represents a specific weight

Decimal	8421	5421	2421	5211
0	0000	0000	0000	0000
1	0001	0001	0001	0001
2	0010	0010	0010	0011
3	0011	0011	0011	0101
4	0100	0100	0100	0111
5	0101	1000	1011	1000
6	0110	1001	1100	1010
7	0111	1010	1101	1100
8	1000	1011	1110	1110
9	1001	1100	1111	1111

For example, in 8421BCD code, 1001 the weights of 1, 0, 0, 1 (from left to right) are 8, 4, 2 and 1 respectively. The codes 8421BCD, 2421BCD, 5211BCD are all weighted codes.

eighted codes: The non-weighted codes are not positionally weighted. In other words, each digit position within the number is not assigned a fixed value (or weight).

Examples are

- xcess-3
- ray code

DECIMAL	EXCESS - 3	GRAY CODE
0	0011	0000
1	0100	0001
2	0101	0011

6.1.3 EXCESS – 3 CODES:-

- This is another form of BCD code, in which each decimal digit is coded into a 4-bit binary code.
- The code for each decimal digit is obtained by adding decimal 3 to the natural BCD code of the digit.

GRAY CONVERSION:-

- Record the mostsignificant bit add the binary MSB to the next significant bit of the

Gray code.

- Record the result, ignoring carrier continue the process, until the LSB is reached.

REFLECTIVE CODES: A code is reflective when the code is self-complementing. In otherwords, when the code for 9 is the complement the code for 0, 8 for 1, 7 for 2, 6 for 3 and 5 for 4. 2421BCD, 5421BCD and Excess-3 code are reflective codes.

SEQUENTIAL CODES: In sequential codes, each succeeding 'code is one binary number greater than its preceding code. This property helps in manipulation of data. 8421 BCD and Excess-3 are sequential codes. ALPHANUMERIC CODES: Codes used to represent numbers, alphabetic characters, symbols and various instructions necessary for conveying intelligible information. ASCII, EBCDIC, UNICODE are the most-commonly used alphanumeric codes.

2. Decimal code

Binary codes for decimal digits require a minimum of four bits. Numerous different codes can be obtained by arranging four or more bits in ten distinct possible combinations. A few possibilities are tabulated.

DECIMAL DIGIT	8421	84-2-1	7421	5421	2421	BIQUINARY
	8 4 2 1	8 4 -2 -1	7 4 2 1	5 4 2 1	2 4 2 1	5 0 4 3 2 1 0
0	0 0 0 0	0 0 0 0	0 0 0 0	0 0 0 0	0 0 0 0	0 1 0 0 0 0 1
1	0 0 0 1	0 1 1 1	0 0 0 1	0 0 0 1	0 0 0 1	0 1 0 0 0 1 0
2	0 0 1 0	0 1 1 0	0 0 1 0	0 0 1 0	0 0 1 0	0 1 0 0 1 0 0
3	0 0 1 1	0 1 0 1	0 0 1 1	0 0 1 1	0 0 1 1	0 1 0 1 0 0 0
4	0 1 0 0	0 1 0 0	0 1 0 0	0 1 0 0	0 1 0 0	0 1 1 0 0 0 0
5	0 1 0 1	1 0 1 1	0 1 0 1	0 1 0 1	1 0 1 1	1 0 0 0 0 0 1
6	0 1 1 0	1 0 1 0	0 1 1 0	0 1 1 0	1 1 0 0	1 0 0 0 0 1 0
7	0 1 1 1	1 0 0 1	1 0 0 0	0 1 1 1	1 1 0 1	1 0 0 0 1 0 0
8	1 0 0 0	1 0 0 0	1 0 0 1	1 0 1 1	1 1 1 0	1 0 0 1 0 0 0
9	1 0 0 1	1 1 1 1	1 0 1 0	1 1 0 0	1 1 1 1	1 0 1 0 0 0 0

Error detection code

In data transmission, Interference and physical defects in the communication medium can cause random bit errors. As the signal is transmitted through a media, the signal gets corrupted because of noise and distortion. Therefore the media is not reliable. To achieve a reliable communication through this unreliable media, there is need for detecting the error in the signal so that suitable mechanism can be devised to take corrective actions.

Error coding is a method of detecting and correcting these errors to ensure information is

transferred intact from its source to its destination

The errors can be divided into two types:

• Single-bit Error: only one bit of given data unit (such as a byte, character, or data unit) is changed from 1 to 0 or from 0 to 1.

• Burst Error: two or more bits in the data unit have changed from 0 to 1 or vice-versa. (Here doesn't necessary means that error occurs in consecutive bits)

Error Detecting Codes:

• dimensional Parity check
• Checksum
• Cyclic redundancy check

Detecting Errors using simple parity check

Suppose we are transmitting 7-bit ASCII characters. A parity bit is added to each character to make it 8 bits. Parity can detect all single-bit errors

–If even parity is used and a single bit changes, it will change the parity to odd, which will be detected at the receiver end

–The receiver end can detect the error, but cannot correct it because it does not know which bit is erroneous Parity can also detect some multiple-bit errors

Table 1 shows the four bit data word and its corresponding code words

Decimal value	Data block	Parity bit	Code word
0	0000	0	00000
1	0001	1	00011
2	0010	1	00101
3	0011	0	00110
4	0100	1	01001
5	0101	0	01010
6	0110	0	01100
7	0111	1	01111
8	1000	1	10001
9	1001	0	10010
10	1010	0	10100
11	1011	1	10111
12	1100	0	11000
13	1101	1	11011
14	1110	1	11101
15	1111	0	11110

3. Gray Code- Reflection and Self Complementary codes

- Gray Code is a non-weighted code which belongs to a class of codes called minimum change codes.
- Gray Code is an alternative binary representation, devised such that, between any two adjacent numbers, *only one bit* changes at a time.

Binary	Dec	Gray
00000	0	00000
00001	1	00001
00010	2	00011
00011	3	00010
00100	4	00110
00101	5	00111
00110	6	00101
00111	7	00100
01000	8	01100
01001	9	01101
01010	10	01111
01011	11	01110
01100	12	01010
01101	13	01011
01110	14	01001
01111	15	01000

- To the left we see three columns of data. These are representations of the same numbers 0-15 in different ways.
    - In the middle is the decimal value.
    - On the left is positional notation binary
    - On the right is Gray code.
- You will notice that, on the right, each adjacent row is different from it's neighbours by no more than one bit.
- The term Gray code is often used to refer to a "reflected" code, or more specifically still, the binary reflected Gray code.

Complementary Code

A code is said to be self-complementary if the code for 9's complement of N i.e. 9-N can be obtained by interchanging all 0s and 1s.

Decimal 9 is the complement of code for 0, 8 for 1, 7 for 2 and so on.

For a code to be self complementing, the sum of all its weights must be 9. digit.8421 and 5421 codes are not self complementing codes whereas 5211,2421,3321, 4321 are self complementing.

In general, a code is self-complementary if we produce a code by taking the first complement of the digit which is same as 9's complement of the number.

Reflective code

- Imaged about the centre entries with one bit changed
- Example ï 9ís complement of a reflected BCD code word is formed by changing only one of its bits
- In the Gray code example shown below, the MSB bit alone is changing and the remaining bits is reflected mirror image about the centre. For clarity, the MSB is removed.
- Gray code Reflected property of Gray code

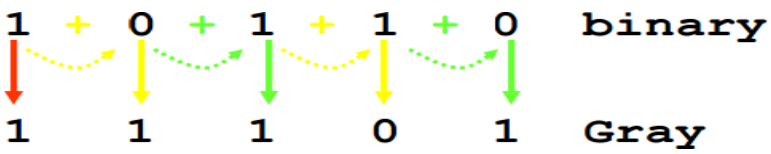
```
0000
0001
0011
0010
0110
0111
0101
0100
1100
1101
1111
1110
1010
1011
1001
1000
```

```
x000
x001
x011
x010
x110
x111
x101
x100
-------- mirror
x100
x101
x111
x110
x010
x011
x001
x000
```

Binary-to-Gray code conversion

- The MSB in the Gray code is the same as corresponding MSB in the binary number.
- Going from left to right, add each adjacent pair of binary code bits to get the next Gray code bit.
- Discard carries.

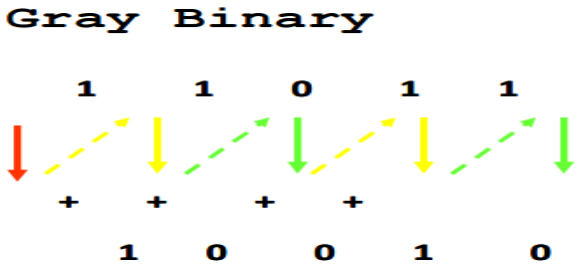
Problem: Convert 10110 to gray code



Gray-to-Binary Conversion

- The MSB in the binary code is the same as the corresponding bit in the Gray code.
- Add each binary code bit generated to the Gray code bit in the next adjacent position.
- Discard carries.

Problem: Convert the Gray code word 11011 to binary



Binary-Coded Decimal Code

Although the binary number system is the most natural system for a computer because it is readily represented in today's electronic technology, most people are more accustomed to the decimal system. One way to resolve this difference is to convert decimal numbers to binary, perform all arithmetic calculations in binary, and then convert the binary results back to decimal. This method requires that we store decimal numbers in the computer so that they can be converted to binary. Since the computer can accept only binary values, we must represent the decimal digits by means of a code that contains 1's and 0's. It is also possible to perform the arithmetic operations directly on decimal numbers when they are stored in the computer in coded form.

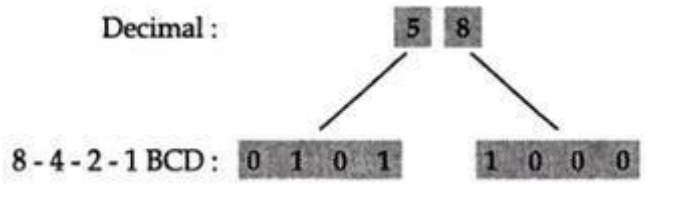
A binary code will have some unassigned bit combinations if the number of elements in the set is not a multiple power of 2. The 10 decimal digits form such a set. A binary code that distinguishes among 10 elements must contain at least four bits, but 6 out of the 16 possible combinations remain unassigned. Different binary codes can be obtained by arranging four bits into 10 distinct combinations. This scheme is called *binary-coded decimal* and is commonly referred to as BCD.

A number with k decimal digits will require $4k$ bits in BCD. Decimal 396 is represented in BCD with 12 bits as 0011 1001 0110, with each group of 4 bits representing one decimal digit. A decimal number in BCD is the same as its equivalent binary number only when the number is between 0 and 9. A BCD number greater than 10 looks different from its equivalent binary number, even though both contain 1's and 0's. Note that the BCD code is not self-complementing. Moreover, the binary combinations 1010 through 1111 are not used and have no meaning in BCD. Consider decimal 185 and its corresponding value in BCD and binary:

$(185)_{10} = (0001\ 1000\ 0101)_{BCD} = (10111001)_2$

Decimal	BCD Code			
Digit	8	4	2	1
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1

Table 1 in multi digit BCD coding



REFERENCES:-

1. Brian W.Kernighan and Dennis Ritchie,C Programming Langauge,Prantice Hall of India.
2. Yashwant Kanetkar,Let US C,BPB Publication.

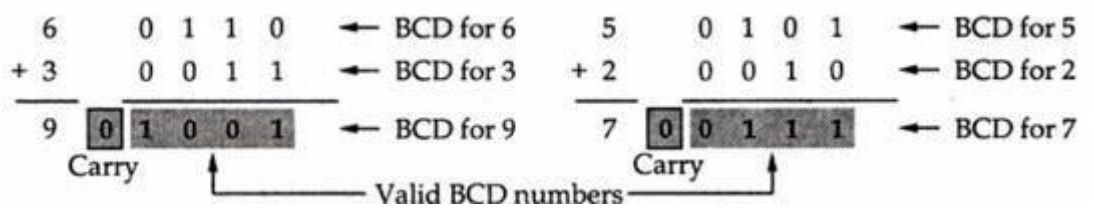
LECTURE -10 (UNIT-01)

Addition:

The addition of two BCD numbers can be best understood by considering the three cases that occur when two BCD digits are added.

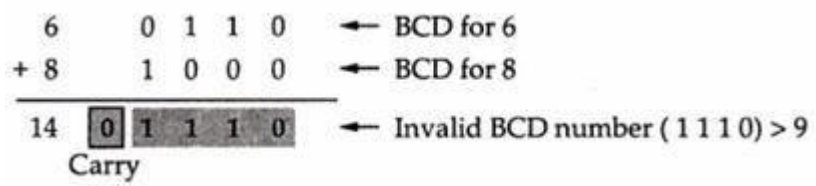
Sum equals 9 or less with carry 0

Let us consider additions of 3 and 6 in BCD.

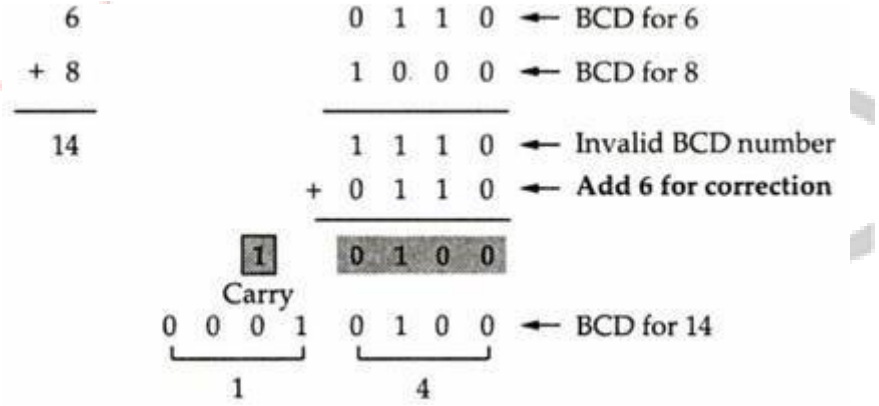


Sum greater than 9 with carry 0

Let us consider addition of 6 and 8 in BCD

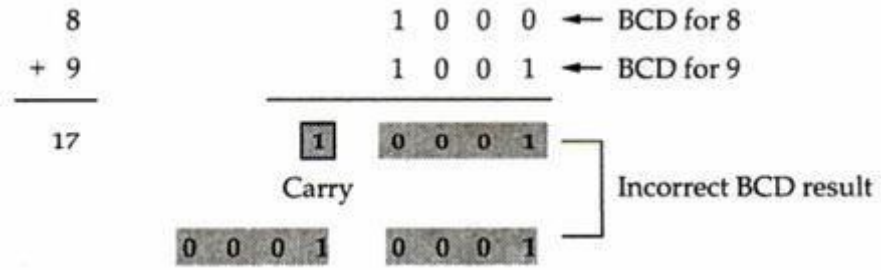


The sum 1110 is an invalid BCD number. This has occurred because the sum of the two digits exceeds 9. Whenever this occurs this occurs the sum has to be corrected by the addition of six (1110) in the invalid BCD number, as shown below

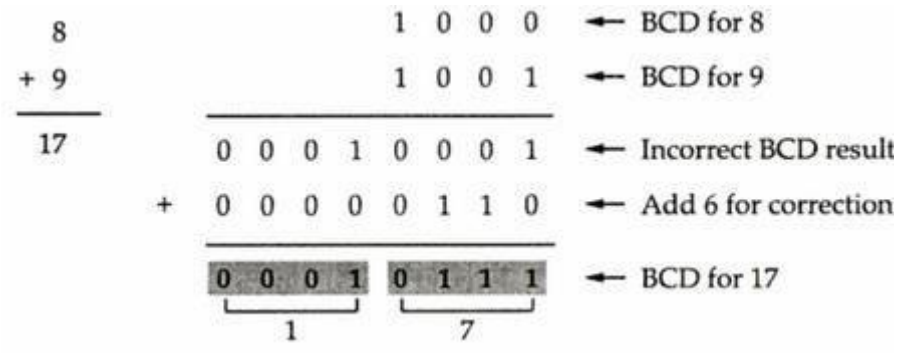


Sum equals 9 or less with carry 1

Let us consider addition of 8 and 9 in BCD



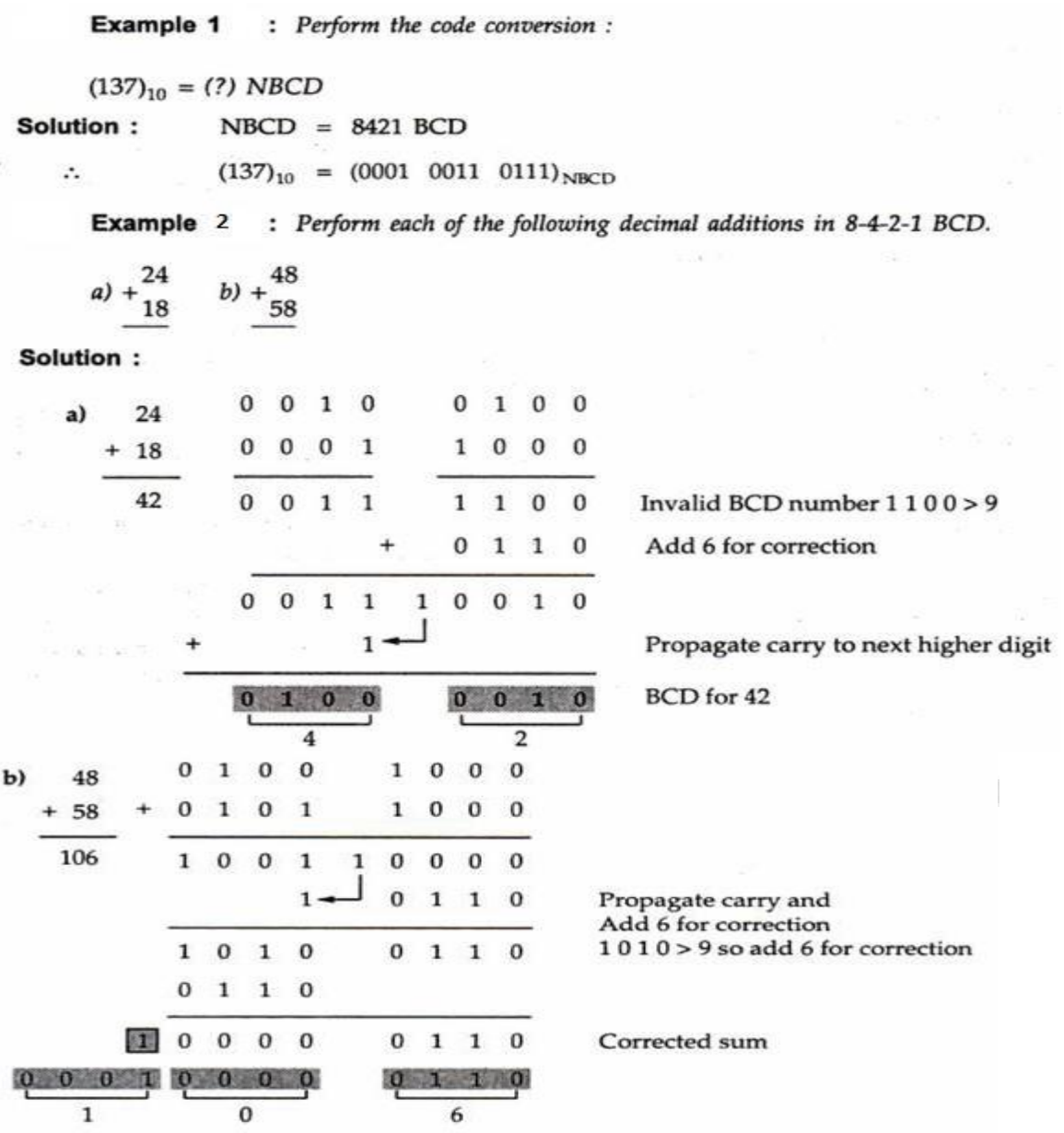
In this case, result (001 0001) is valid BCD number, but it is incorrect. To get the correct BCD result correction factor of 6 has to be added to the least significant digit sum, as shown.



BCD addition procedure

1. Add two BCD numbers using ordinary binary addition.
2. If four bit sum is equal to or less than 9, no correction is needed. The sum is in proper BCD form.

3. If the four bit sum is greater than 9 or if a carry is generated from the four-bit sum,



the sum is invalid.To correct the invalid sum, add 0110_2 to the four-bit sum. If a carry results from this addition, add it to the next higher-order BCD digit.

Alphanumeric codes

Alphanumeric codes are sometimes called character codes due to their certain properties. Now these codes are basically binary codes. We can write alphanumeric data, including data, letters of the alphabet, numbers, mathematical symbols and punctuation marks by this code which can be easily understandable and can be processed by the

computers. Input output devices such as keyboards, monitors, mouse can be interfaced using these codes. 12-bit Hollerith code is the better known and perhaps the first effective code in the days of evolving computers in early days. During this period punch cards were used as the inputting and outputting data. But nowadays these codes are termed obsolete as many other modern codes have evolved. The most common alphanumeric codes used these days are ASCII code, EBCDIC code and Unicode.

Character Code

Many applications of digital computers require the handling not only of numbers, but also of other characters or symbols, such as the letters of the alphabet. For instance, consider a high-tech company with thousands of employees. To represent the names and other pertinent information, it is necessary to formulate a binary code for the letters of the alphabet. In addition, the same binary code must represent numerals and special characters (such as $). An alphanumeric character set is a set of elements that includes the 10 decimal digits, the 26 letters of the alphabet, and a number of special characters. Such a set contains between 36 and 64 elements if only capital letters are included, or between 64 and 128 elements if both uppercase and lowercase letters are included. In the first case, we need a binary code of six bits, and in the second, we need a binary code of seven bits. The standard binary code for the alphanumeric characters is the American Standard Code for Information Interchange (ASCII), which uses seven bits to code 128 characters, as shown in Table below. The seven bits of the code are designated by *b*1 through *b*7, with *b*7 the most significant bit. The letter *A*, for example, is represented in ASCII as 1000001 (column 100, row 0001). The ASCII code also contains 94 graphic characters that can be printed and 34 nonprinting characters used for various control functions.

The graphic characters consist of the 26 uppercase letters (A through Z), the 26 lowercase letters (a through z), the 10 numerals (0 through 9), and 32 special printable characters, such as %, *, and
$.characters. Format effectors are characters that control the layout of printing. They include the familiar word processor and typewriter controls such as backspace (BS), horizontal tabulation (HT), and carriage return (CR). Information separators are used to separate the data into divisions such as paragraphs and pages. They include characters such as record separator (RS) and file separator (FS). The communication-control characters are useful during the transmission of text between remote devices so that it can be distinguished from other messages using the same communication channel before it and after it. Examples of communication-control characters are STX (start of text) and

ETX (end of text), which are used to frame a text message transmitted through a communication channel.

ASCII is a seven-bit code, but most computers manipulate an eight-bit quantity as a single unit called a *byte*. Therefore, ASCII characters most often are stored one per byte. The extra bit is sometimes used for other purposes, depending on the application.

For example, some printers recognize eight-bit ASCII characters with the most significant bit set to

0. An additional 128 eight-bit characters with the most significant bit set to 1 are used for other symbols, such as the Greek alphabet or italic type font.

DEC	OCT	HEX	BIN	Symbol	Description
0	000	00	00000000	NUL	Null char
1	001	01	00000001	SOH	Start of Heading
2	002	02	00000010	STX	Start of Text
3	003	03	00000011	ETX	End of Text
4	004	04	00000100	EOT	End of Transmission
5	005	05	00000101	ENQ	Enquiry
6	006	06	00000110	ACK	Acknowledgment
7	007	07	00000111	BEL	Bell
8	010	08	00001000	BS	Back Space
9	011	09	00001001	HT	Horizontal Tab
10	012	0A	00001010	LF	Line Feed
11	013	0B	00001011	VT	Vertical Tab
12	014	0C	00001100	FF	Form Feed
13	015	0D	00001101	CR	Carriage Return
14	016	0E	00001110	SO	Shift Out / X-On
15	017	0F	00001111	SI	Shift In / X-O

EBCDIC:The EBCDIC stands for Extended Binary Coded Decimal Interchange Code. IBM invented this code to extend the Binary Coded Decimal which existed at that time. All the IBM computers and peripherals use this code. It is an 8 bit code and therefore can accommodate 256 characters. Below is given some characters of EBCDIC code to get familiar with it.

Char	EBCDIC	HEX	Char	EBCDIC	HEX	Char	EBCDIC	HEX
A	1100 0001	C1	P	1101 0111	D7	4	1111 0100	F4
B	1100 0010	C2	Q	1101 1000	D8	5	1111 0101	F5
C	1100 0011	C3	R	1101 1001	D9	6	1111 0110	F6
D	1100 0100	C4	S	1110 0010	E2	7	1111 0111	F7
E	1100 0101	C5	T	1110 0011	E3	8	1111 1000	F8
F	1100 0110	C6	U	1110 0100	E4	9	1111 1001	F9
G	1100 0111	C7	V	1110 0101	E5	blank	...	...
H	1100 1000	C8	W	1110 0110	E6	.	...	...
I	1100 1001	C9	X	1110 0111	E7	(	...	...
J	1101 0001	D1	Y	1110 1000	E8	+	...	...
K	1101 0010	D2	Z	1110 1001	E9	$	...	...
L	1101 0011	D3	0	1111 0000	F0	*	...	...
M	1101 0100	D4	1	1111 0001	F1	)	...	...
N	1101 0101	D5	2	1111 0010	F2	-	...	...
O	1101 0110	D6	3	1111 0011	F3	/		

REFERENCES:-

1. Brian W.Kernighan and Dennis Ritchie,C Programming Langauge,Prantice Hall of India.
2. Yashwant Kanetkar,Let US C,BPB Publication.